

AST-Based Structural Narrowing for Diff Context: A Third Alternative to Raw Hunks and Whole Files

Edgar Herrador
Independent
edgar.herrador@gmail.com
github.com/eherrador

Abstract

Tools that derive training data or review context from code diffs — LLM fine-tuning pipelines built from pull-request history, automated code-review assistants, changelog generators — face a recurring design decision: how much surrounding code to keep around each change. The two dominant choices, the raw unified-diff hunk and the whole file, are both structurally mismatched to the unit of change a developer actually reasons about: the enclosing method, function, class, or field. We present **ast-context-narrowing**, a small library that walks a Tree-sitter AST from a change’s touched byte range upward to the smallest enclosing structural definition, and document six edge-case rules — each backed by a regression test and, in most cases, a concrete production failure that motivated it — required to make that walk reliable across doc comments, blank-line insertions, single-statement fields, and exported class declarations. We evaluate the technique on 175 file changes drawn from 37 real merged pull requests (103 distinct files) in a large, actively maintained open-source TypeScript project (**nestjs/nest**, 76,705 GitHub stars at time of writing). Narrowing resolved a definition node for every evaluated change (175/175), with a median line-count reduction of 71.8% relative to whole-file context — though, as Section 4 discusses, the 100% resolution rate reflects TypeScript’s broad definition-type set rather than evidence that narrowing is always tight; the reduction distribution is in fact bimodal. The library, its tests, and the evaluation harness are released under the MIT license.

1 Introduction

A diff-derived training or review record needs three things to be useful: the change itself, enough surrounding code to make the change legible, and as little *irrelevant* code as possible, since every token of context is something a downstream model (or reviewer) has to read without contributing to the signal being conveyed. Two conventional choices for “enough surrounding code” both miss this balance in opposite directions:

- **The raw diff hunk.** A unified diff pads the actual changed lines with a fixed window of context (conventionally 3 lines) on each side. This window is arbitrary relative to the code’s actual structure: for a short method, it routinely overshoots into a neighboring method or its attributes; for a large method, it badly undershoots, omitting the method’s own signature or the class it belongs to.
- **The whole file.** Always structurally sufficient, but often overwhelmingly so. A one-line bug fix in a 500-line file buries the actual signal in hundreds of unrelated lines — costly for a

human reviewer’s attention, and costly in a much more literal sense for an LLM fine-tuning pipeline, where every token of irrelevant context dilutes the training signal and inflates the sequence length (and therefore memory and compute cost) needed to represent each example.

We argue for a third option: recover the code’s own structural unit of change — the method, function, class, or field the diff actually touched — by parsing the file and walking its AST. This is not a new idea in the abstract (structural/AST-aware diffing has prior art, discussed in Section 5), but we found that a naive implementation of “find the smallest enclosing definition node” degrades silently back into whole-class or whole-file context on real-world diffs unless it specifically accounts for a handful of edge cases. Section 3 documents six such rules, each motivated by a concrete failure mode we observed narrowing a large real-world diff corpus.

Contributions.

1. A small, dependency-light (only Tree-sitter) library implementing AST-based diff-to-structural-context narrowing for four languages (C#, TypeScript, TSX, JavaScript), released as open source.
2. Six documented, individually-tested narrowing rules, each traced to a specific real-world failure mode.
3. An evaluation on real merged-PR diffs from a large open-source project, quantifying both narrowing success rate and context-size reduction relative to whole-file baseline.

2 Problem Statement

Given a source file’s content before a change (**before**), its content after the change (**after**), and a unified diff between them, we want a function

```
narrow(language, before, after, diff) -> (context, output)
```

such that **context** and **output** are the smallest substrings of **before** and **after**, respectively, that (a) are syntactically self-contained (i.e., correspond to one or more complete AST subtrees, not arbitrary line ranges) and (b) fully contain every line the diff touched. When no such substring exists that is smaller than the whole file — e.g., the diff touches only top-level statements outside any function or class, or the file fails to parse — the function should say so explicitly rather than silently returning a degenerate (whole-file or empty) result, so the caller can choose its own fallback.

3 Method

3.1 Baseline algorithm

For each hunk in the diff, convert its touched-line range into a byte range in the corresponding source, then walk the file’s Tree-sitter AST to find the smallest *named* node whose type belongs to a per-language allowlist of “definition” node types (e.g., `method_declaration`, `class_declaration` in C#; `function_declaration`, `method_definition` in TypeScript) and that fully contains the byte range. If a hunk’s range isn’t contained by any node in the allowlist, the smallest enclosing

named node of *any* type is used as a fallback signal that narrowing found no clean structural unit. The byte ranges resolved for every hunk in a file are merged (overlapping/adjacent ranges combined) and their source text concatenated, in source order, as the final `context/output`.

This baseline is a few dozen lines of tree-walking code. It is also, on its own, insufficient — the following six rules were each added after the baseline produced a visibly wrong result on a real diff.

3.2 Rule 1 — Anchor to touched lines, not the padded hunk span

A unified diff hunk’s line range includes a fixed window of unchanged context lines on each side of the actual +/- lines (conventionally 3). For a short method — common in unit test files — this padding alone can span past the method’s own boundaries into a neighboring method or a class-level attribute block. When that happens, *no single node* in the AST contains the padded span, and the baseline algorithm’s containment search has nothing to match at the method level, escalating to the enclosing class.

Fix: re-derive the touched-line range directly from the +/- lines in the hunk body, discarding the padding the unified-diff format adds.

3.3 Rule 2 — Trim trailing statement terminators

Several grammars (JavaScript/TypeScript in particular) model a field or lexical declaration’s trailing `;` as a separate sibling token, not part of the declaration node’s own span. A diff line naturally includes the `;` character. Left untrimmed, an otherwise-exact single-line field edit appears to overrun the field node’s boundary by one byte, which fails the “fully contains” containment check and escalates the match to the enclosing class for what should be a one-line, one-node match.

Fix: after converting a hunk’s line range to a byte range, trim a single trailing `;` (and any whitespace before it) from the end of the range before the containment search.

3.4 Rule 3 — Skip past a preceding doc comment

A JSDoc block or a C# `///` XML-doc summary is parsed as a sibling `comment` node immediately preceding the definition it documents — not as part of that definition’s own span. When a diff hunk rewrites both the doc comment and the body of the method it documents in the same change (a common pattern: updating a method’s behavior alongside its documented contract), the hunk’s touched-line range starts inside the comment node. No single method-shaped node contains a range that starts before its own node begins, so the baseline search escalates to the enclosing class.

Fix: before the containment search, if the range’s start byte falls inside a comment node that is immediately followed (modulo whitespace) by more of the touched range, advance the start byte past the comment. A hunk that touches *only* comment lines is left unmodified, since there is no following code for the range to usefully advance into.

3.5 Rule 4 — A pure insertion at a blank line is a point, not a dropped hunk

A hunk with no - lines (a pure insertion) anchors, on the “before” side, to wherever the insertion occurred in the unchanged file. When that anchor line is blank — a common separator between two statements or methods — naive whitespace trimming of the byte range (needed for Rules 1–2’s correctness) collapses the range to a negative-length span, which a naive implementation drops entirely, silently removing that hunk’s contribution to the narrowed context.

Fix: when trimming collapses a range to zero or negative length, anchor instead on a *zero-width point* at the original (untrimmed) boundary. A point still satisfies the containment check used by the search (`node.start <= point <= node.end`), so it correctly resolves to whichever definition the insertion actually landed inside.

3.6 Rule 5 — Container types (class/struct/interface) are a last resort

A “floating” comment or a blank-line insertion with no enclosing method/property/field around it (e.g., a change to a section-header comment inside a class body, with no adjacent member touched) is, strictly speaking, contained by the whole enclosing class — which is *technically* correct but practically useless as narrowed context: a one-line comment edit should not pull in an entire multi-hundred-line class.

Fix: partition each language’s definition-type allowlist into “member” types (methods, functions, fields, properties) and “container” types (classes, structs, interfaces). The containment search tries the smallest enclosing node of *any* type first (including unnamed/anonymous nodes), then the smallest enclosing member type, and only falls back to a container type if no smaller candidate of either kind exists.

3.7 Rule 6 — An exported container is still a container

TypeScript/JSEX register `export_statement` itself as a member-level definition type, because a bare `export function foo() {}` or `export const x = ...` — where the export wraps a non-container declaration — genuinely is the smallest sensible unit of context for a top-level exported declaration. But `export class Foo { ... }` parses as an `export_statement` node *wrapping* a `class_declaration`. Without special-casing this, such a node satisfies the member-type allowlist (since `export_statement` is registered as one), sidestepping Rule 5’s container demotion entirely — a floating comment inside an exported class would escalate to the whole exported class instead of falling through to a smaller node.

Fix: at containment-check time, treat an `export_statement` as a container (subject to Rule 5’s last-resort treatment) if and only if it directly wraps a node of a container type; otherwise, treat it as a normal member.

4 Evaluation

4.1 Setup

We ran the narrowing algorithm against every qualifying file change in the 100 most recently merged pull requests (as of evaluation time) from [nestjs/nest](#), a large (76,705-star), actively maintained, TypeScript, real-world open source project, via the GitHub REST API. We restricted the sample to file changes where (a) the file’s extension maps to a supported language, (b) the change status is **modified** (excluding wholesale file additions/deletions, which have no meaningful “narrow within the file” case), (c) GitHub reported a non-empty unified diff patch for the file, (d) the change touched fewer than 200 lines (to keep the sample representative of typical, focused changes rather than large mechanical refactors or generated-file updates), and (e) both file revisions were under 60KB (to bound evaluation runtime). This yielded 175 qualifying file changes across 103 distinct files and 37 distinct pull requests. For each we fetched the file’s full content at the PR’s base and head commits and ran `narrow_diff_context`. The evaluation script (`evaluation/run_public_repo_eval.py`) and raw per-file results (`evaluation/results/nestjs_nest.json`) are included in the repository for reproduction.

4.2 Results

Metric	Value
File changes evaluated	175
Narrowing succeeded (structural)	175 (100.0%)
No definition node found (fallback needed)	0
Parse/runtime errors	0
Mean line-count reduction (successful cases)	53.2%
Median line-count reduction (successful cases)	71.8%
Mean byte-count reduction (successful cases)	54.0%
Median byte-count reduction (successful cases)	76.4%

Table 1: Aggregate narrowing results on 175 real file changes from `nestjs/nest`.

Narrowing found *some* definition node for every one of the 175 evaluated changes. We do not read this as “the technique reduces context by 100% of the time” — it means TypeScript’s definition-type allowlist (Section 3, `languages.py`) is broad enough, including module-level `lexical_declaration` and `export_statement`, that almost any top-level change lands inside *some* registered definition type, even if that definition turns out to be most of the file. The more informative signal is the *distribution* of the reduction achieved, which is bimodal rather than tightly clustered around the median:

Roughly 29% of changes narrow to under a tenth of the whole file — typically a small, targeted edit inside one method, a single interface member, or a short utility function in an otherwise large file (the file at the extreme end of this group, `nest-application.interface.ts`, went from 211 lines to 1). At the other end, another 29% see under 10% reduction — these are overwhelmingly cases where the diff itself is genuinely broad (e.g., a mechanical rename or type-signature change touched consistently across a whole file, or a change to a `describe` block that spans most of a test spec file),

Reduction	Share of cases
$\geq 90\%$	28.6%
$\geq 75\%$	49.1%
$\geq 50\%$	58.3%
$< 10\%$	28.6%

Table 2: Distribution of line-count reduction across the 175 evaluated changes.

where a whole-file-sized definition *is* the correct answer, not a failure of narrowing. The median file size in the evaluated sample was 334 lines, so even the “no reduction” tail is not narrowing failing silently on trivial files — it reflects diffs whose true scope actually is most of a substantial file.

4.3 Threats to validity

The evaluation covers a single project and a single (dominant) language (TypeScript); the six rules in Section 3 were each originally motivated by C# and TypeScript diffs from a different, private codebase during the tool’s original development (a small-language-model fine-tuning data pipeline over a large enterprise codebase, described generically to preserve that codebase’s confidentiality), so the rules themselves are not overfit to `nestjs/nest` specifically — but the quantitative success rate and reduction numbers above should not be assumed to transfer unchanged to C#, JavaScript, or TSX-heavy codebases without separate measurement. The 200-line change-size cutoff and 60KB file-size cutoff (Section 4.1) were chosen to keep the evaluation’s runtime and API usage reasonable, not derived from any principled threshold; a broader sample is future work.

5 Related Work

Structural/AST-aware diffing has a long history — GumTree and similar tools compute fine-grained AST-level edit scripts between two versions of a file, primarily for precise change *detection and classification* [1] rather than context *narrowing* for a downstream consumer. Our problem is narrower and more specific: given that a change already exists (as a unified diff, the near-universal interchange format for VCS tooling), select the smallest well-formed structural region of the file that contains it. This is closer in spirit to IDE “go to enclosing symbol” navigation than to edit-script computation, applied programmatically at data-set or context-window construction time rather than interactively.

In LLM fine-tuning and retrieval-augmented code assistance specifically, context construction from diffs is usually described only at the level of “we extract the relevant function/class” without publishing the edge-case handling that makes such extraction reliable across real-world diffs — the six rules in Section 3 are, to our knowledge, not documented elsewhere as a checklist, despite (based on our own experience discovering them one production failure at a time) being generic to any Tree-sitter-based narrowing implementation rather than specific to our original use case. The parsing itself is built on Tree-sitter [2], an incremental parsing library providing concrete syntax trees for many languages.

6 Limitations

- **No hybrid-grammar support.** Razor (`.cshtml`), Vue single-file components, and similar host/guest-language files are not supported; narrowing them needs dedicated handling of the embedding boundary, which we leave to future work.
- **Definition-type allowlists are manually curated per language.** Extending support to a new language requires identifying the right Tree-sitter node types by hand; we have not attempted to derive this automatically from a grammar.
- **Single-file scope.** The algorithm narrows within one file at a time and does not attempt cross-file context (e.g., pulling in a called function’s definition from another file), which some use cases may still need on top of this technique.
- **Language-, not natural-language-, specific.** The technique’s only dependency on the underlying grammar is syntactic; it makes no natural-language assumptions about identifiers or comments.

7 Conclusion

Whole-file and raw-diff-hunk context are both structurally mismatched to how developers actually reason about a change. AST-based narrowing recovers the code’s own unit of change, but only reliably once a handful of specific, individually-motivated edge cases are handled. We release `ast-context-narrowing`, implementing and testing all six rules, under the MIT license, along with the evaluation harness used in Section 4, so the technique and its evaluation are both reproducible and extensible to new languages and codebases.

Availability

Code: <https://github.com/eherrador/ast-context-narrowing> (MIT license)

References

- [1] Jean-Rémy Falleri, Floréal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus. Fine-grained and accurate source code differencing. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering, ASE '14*, pages 313–324, New York, NY, USA, 2014. ACM.
- [2] Tree-sitter contributors. Tree-sitter: An incremental parsing system for programming tools. <https://tree-sitter.github.io/tree-sitter/>. Accessed 2026-09-22.